

AAES-OS Invariant Verification EngineSpecification v1.0

Document ID:	AAES-OS-IVE-SPEC-001
Version:	1.0 — Final
Date:	June 17, 2026
Classification:	Canonical System Artifact — Constitutional Layer
Authors:	Jon (Architecture) Dar-z (Governance) Daniel (Engineering)
Status:	ACTIVE — Pending Constitutional Signature
Supersedes:	None (Initial Issue)
Review Cycle:	On structural doctrine change or invariant revision

Table of Contents

- 1. Preamble and Purpose
- 2. Governing Architecture
 - 2.1 Master Control Equation
 - 2.2 Governance Hierarchy
 - 2.3 IVE Stack Position
 - 2.4 D-3 Tri-Core Continuity Domains
- 3. IVE Input / Process / Output Model

- 3.1 Input Payload
 - 3.2 Verification Pipeline
 - 3.3 Output
 - 4. The Six Invariant Families — Formal Definitions and Runtime Predicates
 - 4.1 Identity Invariant (AAIS-1 / Art. VIII)
 - 4.2 Boundary Invariant (AAIS-2 / V-INV-3)
 - 4.3 Continuity Invariant (AAIS-3)
 - 4.4 Causality Invariant (AAIS-4 / V-INV-4)
 - 4.5 Mutation Invariant (AAIS-5 / Doctrine XIV / Doctrine XV)
 - 4.6 Preservation Invariant (Doctrine XIV / Art. IX / Meta-Invariant)
 - 5. RSL Safety Gate Integration
 - 6. Decision Receipt Schema
 - 7. IVE Failure Modes and Recovery Protocol
 - 8. Open Engineering Tasks (Roadmap)
 - 9. Constitutional Signatures
-

1. Preamble and Purpose

The **AAES-OS Invariant Verification Engine** (hereafter **IVE**) is the runtime component of the AAES-OS governance framework responsible for converting written governance law into executable, mechanically enforced invariant checks. It is not a recommendation engine. It is not an advisory system. The IVE is a hard enforcement boundary: every governed state transition in the AAES-OS system must pass through the IVE before execution is admitted. There is no bypass path, no emergency override, and no exception process that circumvents IVE verification.

The IVE exists because abstract governance principles — however well-authored — are inert without a runtime mechanism capable of instantiating them as checkable conditions at the moment of execution. The gap between a written invariant and an

enforced invariant is the gap between a constitutional document and a functioning constitution. The IVE closes that gap.

*"A system may improve itself. A system may optimize itself. A system may extend itself.
A system may never redefine the laws that govern those activities."*

The central engineering insight that motivated the IVE design is the distinction between **change** and **valid change**. Static systems do not require governance infrastructure. A system with a fixed state space, fixed behavior, and no capacity for evolution cannot violate its own invariants — it simply operates until it fails. Governance is required precisely because the AAES-OS is a *dynamic, self-extending, multi-agent system* capable of restructuring its own operations, expanding its own capabilities, and spawning subordinate agents with delegated authority.

A dynamic system without enforceable invariants is a system without a constitutional boundary. It may improve itself into something unrecognizable. It may optimize itself in directions that contradict its mission. It may extend itself in ways that break the causal traceability required for accountability. The IVE prevents this not by prohibiting change, but by *governing the geometry of change*.

"Continuity is not 'the system stays the same.' Continuity is 'the system can change while remaining a valid descendant of its previous state.'"

The IVE operationalizes this definition. A state transition is **valid** if and only if the resulting state can be demonstrated to be a lawful descendant of the previous state — meaning it satisfies all invariant predicates, falls within the authorized mutation boundary, maintains causal traceability, and preserves the governance layer structure. Any transition that cannot be so demonstrated is **rejected**.

This document constitutes the complete formal specification of the IVE. It defines the IVE's inputs, its verification pipeline, the six invariant families it enforces, the output artifacts it produces, its integration with the RSL safety gate, its failure behavior, and its recovery protocol. Upon execution of constitutional signatures in Section 9, this document becomes a **Constitutional Artifact** of the AAES-OS governance framework and may not be altered except through the constitutional amendment process defined in Doctrine XIV.

2. Governing Architecture

2.1 Master Control Equation

The AAES-OS system's state evolution is governed by the following **Master Control Equation**, which defines the only lawful mechanism by which system state may advance from one time step to the next:

$$S(t+1) = F(S(t), \text{Proj}\Omega(\text{Cortex}(A(t), I(t))), \text{RSL})$$

Subject to the following simultaneous constraints, all of which must hold at every time step t :

- **$P'(t) \in \Omega(t)$** — the proposed next state $P'(t)$ must lie within the authorized state manifold $\Omega(t)$ at the time of evaluation
- **$E(t) = \text{RSL}(P'(t))$** — the admitted execution $E(t)$ is exclusively the output of the Runtime Safety Layer applied to the proposed state
- **$\forall t: \exists L(t)$** — for every time step, there must exist a corresponding ledger entry; no state transition may occur without a traceable record

In this equation: **$S(t)$** is the system state at time t ; **$A(t)$** is the set of active agents; **$I(t)$** is the set of active intents; **Cortex** is the cognition layer producing a candidate action; **Proj $_{\Omega}$** is the projection function that constrains the candidate action to the authorized manifold; **RSL** is the Runtime Safety Layer gate; and **F** is the governed state transition function. The IVE is the mechanism that enforces Proj $_{\Omega}$ and validates all three constraint conditions before admitting execution.

2.2 Governance Hierarchy

The AAES-OS governance stack is organized as a strict, non-invertible hierarchy. Authority and constraint flow exclusively downward. No lower layer may assert authority over, modify, reinterpret, or override a higher layer. The hierarchy is:

Layer	Level	Description	Mutability
Law	1 (Apex)	Constitutional invariants, doctrines, and governing articles. The foundational constraint layer.	Constitutional amendment only
Blueprint	2	Architectural doctrine and structural design principles derived from Law.	Tri-Core approval required
Contract	3	Formal behavioral agreements between agents, layers, and subsystems.	Authorized governance review
Implementation	4	Concrete engineering realizations of contracted behaviors.	Engineering authority with evidence
Pipeline	5	Execution workflows, scheduled processes, and operational sequences.	Operational authority with audit trail
Tool	6 (Leaf)	Individual executable functions, capabilities, and atomic actions.	Capability-bounded execution

Invariant Constraint

The governance hierarchy must never be inverted. Any action, modification, or system state that results in a lower-layer artifact asserting authority over a higher-layer artifact constitutes a critical invariant violation and triggers immediate ESCALATE response from the IVE.

2.3 IVE Stack Position

Within the AAES-OS execution stack, the IVE occupies the **gate position between the Intent layer and the Execution layer**. This position is not incidental — it is the only architecturally valid location for a universal enforcement boundary. An enforcement mechanism placed at any other position in the stack would be bypassable through alternative execution paths.

Every governed state transition is routed through the IVE. The routing is not optional, not configurable at the agent level, and not subject to capability-based override. The flow is:

Stage	Layer	Description
1	Intent Formation	Agent or system process generates a proposed state transition
2	IVE Gate	All six invariant families are checked; decision receipt is produced
3	RSL Safety Gate	Risk score is computed; admission or denial is finalized
4	Execution	Admitted transitions proceed; denied transitions are logged and escalated
5	Ledger Record	Immutable record is created; evidence artifacts are stored

2.4 D-3 Tri-Core Continuity Domains

The IVE enforces continuity across three formally distinct domains, each owned by one member of the constitutional authorship team. This structure, designated the **D-3 Tri-Core**, ensures that continuity is assessed from three independent and complementary perspectives before any transition is admitted. All three domains must pass. Failure in any domain constitutes failure of the continuity invariant.

Domain	Owner	Question Answered	Scope
Architecture Continuity	Jon	Does the system remain aligned with original doctrine?	Structural integrity, layer relationships, doctrine fidelity
Governance Continuity	Dar-z	Does expansion preserve mission, boundaries, and commercial coherence?	Mission alignment, boundary enforcement, commercial validity
Engineering Continuity	Daniel	Does implementation preserve technical integrity and execution reliability?	Code integrity, execution reliability, dependency soundness

3. IVE Input / Process / Output Model

The IVE operates as a deterministic verification pipeline. Given a fully specified input payload, the IVE produces a deterministic output. There is no probabilistic behavior, no partial admission, and no cached result reuse across distinct transitions. Every verification call is atomic and complete.

3.1 Input Payload

Every IVE verification call must supply a complete **Input Payload**. A payload with any missing required field is treated as malformed and results in automatic DENY without entering the verification pipeline. The required fields are:

Field	Type	Description	Required
previous_state	Serialized snapshot	Complete, hash-verifiable snapshot of system state at time t	Yes
proposed_transition	Structured operation object	The operation, mutation, or action being requested; must include action type, affected artifacts, and origin agent	Yes
authority_context	Identity/capability profile	Identity, role, and capability ceiling of the requesting agent; must be verifiable against the registered authority registry	Yes
evidence_bundle	Artifact collection	Existing audit artifacts, lineage chain, and prior decision receipts that establish the causal history of the requesting agent and transition	Yes

3.2 Verification Pipeline

Upon receipt of a valid input payload, the IVE executes the following sequential verification pipeline. All five steps must pass. The pipeline is not parallelized: steps execute in order, and a failure at any step halts the pipeline and produces an

INVALID TRANSITION output immediately. There is no partial credit and no conditional admission.

Step 1 – Validate Previous State

The `previous_state` snapshot is validated against all invariant predicates. This confirms that the system was in a lawful state before the transition was requested. A transition from an already-invalid state is itself invalid, regardless of the proposed next state.

Step 2 – Validate Proposed Next State

The state that would result from the proposed transition is constructed and validated against all invariant predicates. This confirms that the destination state is lawful — that it lies within the authorized state manifold $\Omega(t)$.

Step 3 – Verify the Transition Path

The path connecting the previous state to the proposed next state is verified for geometric validity (state manifold membership), calculus validity (bounded rate of change), and phase validity (directional continuity). A valid start and a valid end connected by an invalid path constitutes an invalid transition.

Step 4 – Confirm Allowed Mutation

The mutation boundary is checked: does the proposed transition touch any protected artifacts? If so, does the requesting agent hold constitutional authority? Lineage continuity is confirmed: does the requesting agent's evidence bundle establish valid causal descent from prior authorized states?

Step 5 – Produce Decision Receipt

Regardless of pass or fail outcome, a complete, immutable **Decision Receipt** is produced and committed to the ledger. The receipt records all intermediate results, the composite verdict, and all evidence artifacts generated during verification. Execution of Step 5 is not conditional on the outcome of Steps 1–4.

3.3 Output

The IVE produces one of two composite verdicts:

Verdict	Condition	Consequences
VALID CONTINUATION	All five pipeline steps pass; all six invariant families return PASS	Execution admitted; decision receipt produced and committed to ledger; transition proceeds to RSL gate
INVALID TRANSITION	Any pipeline step fails; any invariant family returns FAIL	Execution blocked; violation record produced; escalation triggered per invariant-specific response protocol; rollback evaluated

4. The Six Invariant Families — Formal Definitions and Runtime Predicates

The IVE enforces six formally distinct invariant families. Each family represents a different dimension of system lawfulness. Collectively, they constitute the complete set of conditions that must hold at all times for the AAES-OS system to be in a valid constitutional state. No invariant family is subordinate to another; all six carry equal enforcement weight.

For each invariant family, this section provides: the formal definition, the core rule, the runtime predicate in Python pseudocode, the violation response, and the evidence artifact produced.

4.1 Identity Invariant AAIS-1 Art. VIII

Formal Definition

An agent cannot become what it was not authorized to be. The **capability boundary** is the identity boundary. An agent's identity is not merely its name or its registered identifier — it is the complete, bounded set of roles and capabilities that were granted to it at the time of its authorization. Any expansion of that set, however incremental, constitutes a change of identity that must be separately authorized. Unauthorized identity expansion is not tolerated in any form, including through capability accumulation, role inheritance exploitation, or indirect delegation chains.

Core Rule

No identity expansion may occur outside the agent's authorized profile as registered in the authority registry at the time of the transition request.

Runtime Predicate

```
def identity_check(agent, authority_context):    return (    agent.role    in authority_context.authorized_roles        and agent.capabilities <= data-id="313" authority_context.capability_ceiling        and agent.identity_hash    == authority_context.registered_hash    )
```

All three conditions must evaluate to `True` simultaneously. The identity hash comparison is performed using SHA-256. A mismatch in any single field returns `False`.

Violation Response

DENY + log `identity_violation` event + notify governance layer. The agent is not admitted to execution. The violation is recorded with the agent's claimed identity, the authority context against which it was checked, and the specific field that failed.

Evidence Artifact Produced

`identity_assertion_receipt` — fields: `agent_id`, `timestamp`, `authorized_roles`, `capability_ceiling`, `identity_hash_match`, `result` (PASS | FAIL).

4.2 Boundary Invariant AAIS-2 V-INV-3

Formal Definition

Agents operate only inside admitted behaviors. There are no **hidden authority expansions**, no unregistered role changes, and no unauthorized execution paths. The admitted action space is a formally defined set; all agent actions must be demonstrably projectable into that space. The formal expression of this constraint is:

$$E(t) \in \text{Substrate_Action_Space}$$

where $E(t)$ is the set of all actions executed at time t and *Substrate_Action_Space* is the registered set of admitted behaviors for the requesting agent's capability profile. This invariant is enforced geometrically: the proposed action is projected into the Universal Language (UL) geometry, and the projection must fall within the admitted action space.

Core Rule

Every executed action must be demonstrably within the admitted action space for the requesting agent's current, registered capability profile.

Runtime Predicate

```
def boundary_check(action, admitted_action_space, ul_geometry):    projected
= project_into_ul(action, ul_geometry)    return projected in
admitted_action_space
```

The `project_into_ul` function applies the UL geometric projection to normalize the action into a canonical form for comparison. The `admitted_action_space` is retrieved from the registered authority registry at the time of verification — it is never cached from a prior call.

Violation Response

Hard block + quarantine of requesting agent + governance alert. The agent is suspended from further execution pending governance review. The quarantine is not lifted by the IVE — it requires explicit governance action.

Evidence Artifact Produced

`boundary_assertion_receipt` — fields: `action_id`, `ul_projection_result`, `admitted_space_version`, `projection_match`, `result` (PASS | FAIL).

4.3 Continuity Invariant AAIS-3

The Continuity Invariant is the most mathematically complex of the six invariant families. It is evaluated across three independent mathematical views — geometric, calculus-based, and trigonometric — and all three must pass for the invariant to be satisfied. A PASS from two views and a FAIL from one is a FAIL of the Continuity Invariant. There is no weighted scoring.

View 1 — Geometry: Valid State Manifold

The system evolves through a **state space** that has a formally defined valid region — the authorized state manifold Ω . The geometric view of continuity asserts that the system may move through state space freely, but it may never leave the valid region. A state outside Ω is not merely suboptimal; it is constitutionally unlawful. Both the previous state and the proposed next state must be valid members of the manifold.

```
def state_is_valid(state, constraints, tol=1e-9):    return
all(abs(rule(state)) < tol for rule in constraints) def
continuity_geometry_check(previous_state, next_state, constraints):
return (    state_is_valid(previous_state, constraints)    and
state_is_valid(next_state, constraints)    )
```

The `constraints` list contains all active invariant predicate functions. The tolerance `tol=1e-9` accommodates floating-point precision without permitting material violation. The constraint set is versioned and must match the version registered in the authority context.

View 2 — Calculus: Controlled State Evolution

A governed system may not **teleport** between states. State teleportation — a sudden, discontinuous jump from one state to another without a bounded, traceable path — is a signal of corruption, unauthorized mutation, or broken causality. The calculus view enforces that the magnitude of state change between t and $t+1$ is bounded by the maximum authorized change rate for the transition type being requested.

```
def continuity_calculus_check(previous_state, next_state, maximum_change):  
    return distance(previous_state, next_state) <= maximum_change
```

The `distance` function computes the appropriate norm for the state space (Euclidean distance for continuous spaces; Hamming distance for discrete state vectors). The `maximum_change` parameter is derived from the transition type's registered change boundary and may not be set by the requesting agent.

View 3 — Trigonometry: Phase and Direction Preservation

The **direction** of system evolution cannot suddenly and unjustifiably flip. This view applies to policy direction, decision trajectory, optimization path, and agent coordination vector. An abrupt reversal of direction — without a recorded, justified reason — is a signal of governance compromise, external interference, or internal corruption. The phase continuity check ensures that the angular change in system direction between time steps does not exceed the maximum authorized phase delta for the current operational context.

```
def phase_continuity(old_phase, new_phase, max_delta):  
    delta = normalize_angle(new_phase - old_phase)  
    return abs(delta) <= max_delta
```

The `normalize_angle` function maps the angular difference to the interval $[-\pi, \pi]$ to handle wraparound correctly. The `max_delta` is a governance-registered parameter; it may be relaxed for explicitly authorized strategic pivots but only through the constitutional amendment process.

Unified Continuity Check

All three views must return True for the Continuity Invariant to pass:

```
def continuity_invariant_check(previous_state, next_state, old_phase,  
                               new_phase, constraints, maximum_change,  
                               max_phase_delta):  
    geometry = continuity_geometry_check(previous_state, next_state, constraints)  
    calculus = continuity_calculus_check(previous_state, next_state, maximum_change)  
    phase = phase_continuity(old_phase, new_phase, max_phase_delta)  
    return geometry and calculus and phase
```

Violation Response

ESCALATE + freeze execution lane + supervisor notification + lineage review.

The Continuity Invariant violation triggers the highest-severity response because it may indicate systemic compromise rather than a localized error. The execution lane is frozen until supervisor review is complete.

Evidence Artifact Produced

`continuity_verification_receipt` — fields: `geometry_result`, `calculus_result`, `phase_result`, `composite_verdict`, `constraint_set_version`, `maximum_change_bound`, `max_phase_delta`, `timestamp`.

4.4 Causality Invariant AAIS-4 V-INV-4

Formal Definition

Every meaningful system action must leave a traceable artifact. The causal chain is not optional — it is the evidence infrastructure upon which all governance review, rollback capability, and accountability architecture depends. The formal expression is:

$$\forall S(t) \rightarrow \exists L(t)$$

For every system state $S(t)$, there must exist a corresponding ledger entry $L(t)$.

There are no **silent state mutations**. A state change that does not produce a ledger record is not a valid state change — it is a governance violation, regardless of whether the resulting state is otherwise lawful. The absence of a record is itself the violation.

Core Rule

Action implies Record implies Evidence. No transition is valid without a prior ledger commitment that establishes its causal antecedent.

Runtime Predicate

```
def causality_check(transition, ledger):    return (        transition.id        in ledger.records        and ledger.records[transition.id].timestamp <=        transition.execution_time        and        ledger.records[transition.id].evidence_hash is not None    )
```

The temporal ordering constraint (`timestamp <= execution_time`) enforces that the ledger record must have been committed *before or at* the time of execution — not retroactively. The `evidence_hash` must be a valid, non-null SHA-256 hash of the associated evidence bundle.

Violation Response

Quarantine + rollback to last valid ledger state + mandatory audit. The system is returned to the last state for which a valid, complete ledger entry exists. The gap between the last valid ledger entry and the point of violation is flagged for mandatory audit.

Evidence Artifact Produced

`causality_chain_receipt` — fields: `transition_id`, `ledger_entry_id`, `evidence_hash`, `timestamp_ordering_valid`, `chain_depth`, `result` (PASS | FAIL).

4.5 Mutation Invariant AAIS-5 Doctrine XIV Doctrine XV

Formal Definition

The AAES-OS system is designed to adapt. Behavioral adaptation — modifying how the system executes within its governance framework — is not only permitted; it is a design goal. What is not permitted, under any circumstances, is the system **rewriting its own governing law**. The distinction between adapting behavior and amending constitutional rules is the core distinction enforced by the Mutation Invariant.

Protected artifacts — constitutional documents, invariant definitions, governance doctrines, layer relationship specifications, and the IVE itself — may not be modified without constitutional authority, regardless of how the modification is framed. A modification that claims to be an optimization or a refactoring but touches a protected artifact is treated as a constitutional amendment attempt and is subject to the full amendment process.

Core Rule

Adaptation of behavior within the governance framework is lawful. Self-amendment of constitutional rules without constitutional authority is categorically prohibited.

Runtime Predicate

```
def mutation_check(proposed_change, protected_artifacts, authority_context):
    touches_protected = any(
        artifact in
        proposed_change.affected_artifacts
        for artifact in
        protected_artifacts
    )
    if touches_protected:
        return
    authority_context.has_constitutional_authority
    return True
```

The `protected_artifacts` list is itself a protected artifact and may not be modified without constitutional authority. The `has_constitutional_authority` flag is set only through an explicit tri-core constitutional authorization ceremony — it is not a capability that can be delegated or self-asserted.

Violation Response

HARD DENY + production of an immutable violation record + mandatory human escalation. The Mutation Invariant is the only invariant for which the IVE mandates human escalation rather than automated supervisor notification. The rationale is that a mutation violation represents a potential attempt to subvert the constitutional architecture itself, which requires human judgment and authority to assess and respond to.

Evidence Artifact Produced

`mutation_boundary_receipt` — fields: `change_id`, `affected_artifacts`, `protected_artifacts_touched`, `constitutional_authority_present`, `human_escalation_triggered`, `result` (PASS | FAIL).

4.6 Preservation Invariant Doctrine XIV Art. IX Meta-Invariant

Formal Definition

The Preservation Invariant is designated a **Meta-Invariant** because it governs not a single dimension of system state but the integrity of the governance framework

itself. No modification — regardless of its purpose, source, or claimed benefit — may silently weaken any of the following protected dimensions:

- Governance boundaries and layer relationships
- Agent identity and capability ceiling structures
- Evidence chains and causality traceability
- Trust architecture and authority delegation mechanisms
- System invariant definitions (including this invariant)
- The IVE specification and its verification pipeline

The word *silently* is operative. A modification that openly, transparently, and with complete documentation changes one of these dimensions, with a full preservation proof submitted and approved, may be lawful (subject to the mutation invariant). A modification that changes one of these dimensions without adequate disclosure — even if the change is "small" — is a Preservation Invariant violation.

Refactoring, optimization, performance improvement, and structural reorganization are not exempt. Any modification to the system that affects protected dimensions must be accompanied by a complete **Preservation Proof** answering the following five questions:

1. **What changed?** — A complete, specific description of every artifact, behavior, relationship, or constraint that is altered by this modification.
2. **Why?** — The rationale for the modification, including the problem being solved and the alternatives considered.
3. **What remains invariant?** — An explicit enumeration of all invariant properties that the modification leaves unchanged, with verification that this enumeration is complete.
4. **What dependencies are affected?** — A complete dependency map showing all downstream artifacts, agents, and processes that depend on anything being changed.

5. **How is rollback performed?** — A complete, tested rollback procedure that can restore the system to its pre-modification state if the modification is subsequently found to be invalid.

Core Rule

No modification may silently weaken governance boundaries, identity, evidence chains, trust architecture, invariant definitions, or layer relationships. Every modification affecting these dimensions requires a complete preservation proof before admission.

Runtime Predicate

```
def preservation_check(modification, preservation_proof):    return (
preservation_proof.what_changed is not None                and
preservation_proof.rationale is not None                  and
preservation_proof.invariants_unchanged is True           and
preservation_proof.dependency_map is not None              and
preservation_proof.rollback_procedure is not None          )
```

Violation Response

DENY + return of the `preservation_proof_template` to the requesting agent with all missing or incomplete fields flagged. The requesting agent must complete the preservation proof and resubmit. An incomplete proof is not treated as a partial pass — incompleteness in any field is treated as a FAIL for the entire check.

Evidence Artifact Produced

`preservation_assertion_receipt` — fields: `modification_id`,
`proof_completeness_score` (0-5, one point per answered question),
`invariants_confirmed`, `dependency_map_provided`, `rollback_procedure_provided`,
`result` (PASS | FAIL).

5. RSL Safety Gate Integration

The **Runtime Safety Layer** (RSL) is the component that receives the IVE's VALID CONTINUATION verdict and applies a final, risk-score-based admission decision before execution proceeds. The IVE and RSL are distinct components with distinct

responsibilities: the IVE is a binary, predicate-based invariant checker; the RSL is a risk-scored admission gate. Both must pass for execution to be admitted.

The RSL applies the following safety function to the proposed transition P' and current system state S :

$RSL(P', S) =$

ADMIT	if $\rho \leq \theta_{safe}$
DENY	if $\theta_{safe} < \rho \leq \theta_{block} \rightarrow$ drop ops + log violation
ESCALATE	if $\rho > \theta_{block} \rightarrow$ freeze lane + notify governance
WAIT	if dependency unresolved $\rightarrow$ suspend execution

In this function, ρ is the computed risk score for the proposed transition and θ_{safe} and θ_{block} are governance-registered threshold parameters. The RSL does not recheck invariants — that is the IVE's exclusive responsibility. The RSL applies a separate risk assessment over the same transition.

RSL Output	Condition	Effect
ADMIT	$\rho \leq \theta_{safe}$	Execution proceeds; transition is committed; ledger record created
DENY	$\theta_{safe} < \rho \leq \theta_{block}$	Operations dropped; violation logged; requesting agent notified
ESCALATE	$\rho > \theta_{block}$	Execution lane frozen; governance layer notified; supervisor review required
WAIT	Unresolved dependency	Execution suspended; dependency resolution queued; no state mutation

 **Critical Integration Rule**

An IVE violation of any single invariant family is treated as $\rho > \theta_{\text{block}}$ at the RSL level, triggering automatic **ESCALATE** regardless of what a standalone RSL risk computation might yield. The IVE is not advisory to the RSL — an IVE INVALID TRANSITION verdict overrides the RSL's independent risk score. The RSL may only apply its own risk scoring to transitions that have already received IVE VALID CONTINUATION.

6. Decision Receipt Schema

Every IVE evaluation — whether the outcome is VALID CONTINUATION or INVALID TRANSITION — produces a **Decision Receipt**. This receipt is a mandatory, immutable artifact. It is committed to the ledger before the IVE returns its verdict to the calling system. A verification run that does not produce a committed Decision Receipt has not completed — it has failed.

The Decision Receipt serves multiple governance functions: it is the primary evidence artifact for audit reviews; it is the basis for rollback decisions; it is the record against which future transitions can verify lineage continuity; and it is the primary accountability artifact linking a state transition to the agent, authority context, and invariant evaluation that admitted or denied it.

Receipt Fields

Field	Type	Description
receipt_id	UUID	Globally unique identifier for this receipt; generated at receipt creation time
timestamp	ISO 8601	Time at which the IVE completed verification and produced the receipt

Field	Type	Description
previous_state_hash	SHA-256	Hash of the previous state snapshot submitted in the input payload
proposed_transition_id	Reference	Identifier of the requested operation as submitted in the input payload
authority_context_id	Reference	Identifier of the requesting agent's registered authority profile
invariant_results	Object	Structured object containing the result (PASS FAIL) for each of the six invariant families, with Continuity broken into its three sub-results
composite_verdict	Enum	VALID_CONTINUATION or INVALID_TRANSITION
evidence_artifacts	List	References to all six sub-receipts produced during verification
escalation_triggered	Boolean	Whether any invariant violation triggered the ESCALATE response protocol
rollback_required	Boolean	Whether the violation record indicates a rollback to a prior valid state is required
ledger_entry_id	Reference	Reference to the immutable ledger record created when this receipt was committed

Composite Verdict Block — JSON Schema

```
{
  "receipt_id": "ive-receipt-[uuid]",
  "timestamp": "[ISO8601]",
  "previous_state_hash": "[SHA-256 hex string]",
  "proposed_transition_id": "[transition-ref-id]",
  "authority_context_id": "[agent-profile-ref-id]",
  "composite_verdict": "VALID_CONTINUATION | INVALID_TRANSITION",
  "invariant_results": {
    "identity": "PASS | FAIL",
    "boundary": "PASS"
  }
}
```

```
| FAIL",      "continuity": {      "geometry":  "PASS | FAIL",
"calculus":   "PASS | FAIL",      "phase":    "PASS | FAIL",
"composite":  "PASS | FAIL",      },      "causality":  "PASS | FAIL",
"mutation":   "PASS | FAIL",      "preservation": "PASS | FAIL"  },
"evidence_artifacts": [      "identity_assertion_receipt:[ref]",
"boundary_assertion_receipt:[ref]",      "continuity_verification_receipt:
[ref]",      "causality_chain_receipt:[ref]",      "mutation_boundary_receipt:
[ref]",      "preservation_assertion_receipt:[ref]"  ],
"escalation_triggered": true,  "rollback_required": true,
"ledger_entry_id": "[ledger-ref-id]" }
```

① Schema Stability Note

The Decision Receipt schema is a protected artifact under the Mutation Invariant (§4.5). No field may be added, removed, renamed, or retyped without constitutional authority. Consumer systems must be designed to handle the schema as specified here without relying on field presence beyond what is defined above.

7. IVE Failure Modes and Recovery Protocol

The IVE must be designed with explicit, formally specified behavior for every failure mode it may encounter. A verification engine that behaves unpredictably under failure conditions provides no governance guarantee under the very circumstances when governance enforcement is most critical — high-load periods, adversarial conditions, dependency failures, and infrastructure degradation are precisely the conditions under which governance boundaries are most likely to be tested.

The governing principle for all IVE failure modes is: **fail closed**. When in doubt, DENY. When unavailable, DENY. When partially functional, treat the partial failure as a full failure and DENY. The cost of a false denial — a legitimate transition is blocked temporarily — is recoverable. The cost of a false admission — an invalid transition is executed — may not be.

7.1 Failure Mode Specifications

Failure Mode	Trigger Condition	IVE Response	Recovery Path
IVE Unavailable	IVE process is unreachable, crashed, or unresponsive	All transitions default to DENY . No exception. No grace period. No fallback admission path.	Full recovery protocol (§7.2)
Partial Invariant Failure	Any single invariant check cannot complete execution (exception, timeout, data unavailable)	The incomplete check is treated as FAIL . The composite verdict is INVALID TRANSITION.	Resolve data dependency; resubmit complete payload
Ledger Unavailable	The immutable ledger is unreachable or unwritable	All transitions default to WAIT . No state mutation is permitted without a ledger. WAIT is not a time-bounded condition — it persists until the ledger is restored.	Restore ledger connectivity; verify ledger continuity; resume
Authority Context Unavailable	The authority registry cannot be reached to verify the requesting agent's profile	Treat as unauthorized. Default DENY .	Restore authority registry; resubmit with verifiable authority context
Input Payload Malformed	Required payload fields are missing, corrupt, or of incorrect type	DENY without entering verification pipeline. Return payload validation error to caller.	Correct payload; resubmit
Constraint Set Version Mismatch	The constraint set version in the authority context does not match the IVE's current	DENY . Log version conflict. Notify governance.	Synchronize constraint set versions; resubmit

Failure Mode	Trigger Condition	IVE Response	Recovery Path
	registered version		

7.2 IVE Recovery Protocol

The IVE may only return to **ACTIVE** operational status after completing all five stages of the Recovery Protocol in sequence. No stage may be skipped. No stage may be certified by the same component that is recovering. External peer verification is mandatory.

Recovery Stage 1 – Self-Diagnostic Pass

The IVE executes its complete internal self-diagnostic suite. Every invariant predicate function, every input validation routine, every output serializer, and every ledger interface is tested against known-good reference cases. The self-diagnostic must report zero failures before Stage 2 begins.

Recovery Stage 2 – Peer Verification

A peer IVE instance (or an authorized governance peer, if no peer IVE instance is available) independently verifies that the recovering IVE produces correct verdicts for a standard test suite of reference transitions. The peer's verification result is logged as an immutable evidence artifact.

Recovery Stage 3 – Capability Re-Certification

The IVE's capability profile is re-registered with the authority registry. This step confirms that no capability drift, unauthorized modification, or configuration corruption occurred during the failure period. The re-certification is signed by the governance layer.

Recovery Stage 4 – Ledger Continuity Confirmation

The ledger is inspected to confirm that the gap period — the period during which the IVE was unavailable — contains no admitted state transitions. Any state mutations that occurred during the unavailability period without IVE receipts are flagged as causality violations and are subject to mandatory audit before the system can resume normal operation.

Recovery Stage 5 – Supervisor Approval

A designated supervisor (human or authorized governance agent holding supervisor-tier authority) reviews the self-diagnostic report, peer verification result, capability re-certification, and ledger continuity report, and explicitly approves the IVE's return to ACTIVE status. The approval is logged as an immutable ledger artifact.

No Partial Recovery

An IVE that has completed Stages 1–4 but has not received Stage 5 supervisor approval is in **STANDBY** status, not ACTIVE status. A STANDBY IVE may not admit transitions. The transition from STANDBY to ACTIVE is gated exclusively on supervisor approval.

8. Open Engineering Tasks — Development Roadmap

The following items represent the formally identified next development priorities for the IVE and the broader AAES-OS governance framework. These items are not optional enhancements — they are required to bring the IVE to full operational completeness as defined by the constitutional doctrine. Each item should be

tracked as a formal engineering obligation with assigned ownership, evidence requirements, and completion criteria.

#	Task	Description	Priority
1	Stability Analysis	Formally prove convergence of $S(t)$ under the Master Control Equation. Analyze system behavior under oscillation conditions, particularly under constraint tightening scenarios. Define stability bounds and identify the conditions under which the system is guaranteed to remain within Ω .	Critical
2	Failure Injection Engine	Build a controlled simulation environment capable of injecting UL geometry corruption, RSL bypass pressure, and constraint drift. Measure drift amplification across multiple injection scenarios. Results must inform the recovery protocol thresholds and the RSL parameter calibration.	Critical
3	Execution Runtime Specification	Produce a formal specification of the IVE execution runtime: scheduler design, parallel	High

#	Task	Description	Priority
		verification lane control, multi-agent composition rules for simultaneous transition requests from multiple agents, and lane isolation guarantees. This specification must be reviewed as a constitutional artifact before implementation.	
4	AAES-OS Artifact Constitution	Define the metadata envelope required for every artifact produced by the AAES-OS system. The envelope must encode: Identity (who produced it), Governance (under what authority), and Evidence proof status (what invariants were satisfied at production time). All current and future artifacts must be retroactively or prospectively compliant.	High
5	Recovery Verification State Extensions	Add formally specified Recovery Verification State transitions to the Degradation Doctrine. The current Degradation Doctrine specifies	High

#	Task	Description	Priority
		failure states but does not specify the state machine transitions governing movement from degraded states back to operational states under partial recovery scenarios.	
6	Capability Ceiling Lock Implementation	Implement the Capability Ceiling Lock mechanism that prevents runtime accumulation of capabilities beyond the registered ceiling, including through indirect delegation chains. The lock must be enforced at the identity layer, not at the application layer, to prevent circumvention.	High
7	Visual Architecture Diagram	Produce a formal, version-controlled visual representation of the complete AAES-OS execution stack: Substrate → Governance → Cognition → Action → Evidence. The diagram must be a constitutional artifact with its own IVE receipt and preservation proof. Update required upon	Medium

#	Task	Description	Priority
		any layer relationship change.	

9. Constitutional Signatures

This document — **AAES-OS Invariant Verification Engine Specification v1.0**, Document ID AAES-OS-IVE-SPEC-001 — is submitted for constitutional execution. Upon the application of all three signatures below, this document attains the status of a **Constitutional Artifact** of the AAES-OS governance framework, carrying the full force of the constitutional layer.

As Constitutional Artifact, this document:

- May not be modified without constitutional authority as defined in Doctrine XIV
- May not be superseded without the production of a successor document that satisfies a complete Preservation Proof (§4.6) relative to this document
- Must be referenced by IVE version identifier in every Decision Receipt produced by any IVE instance operating under the AAES-OS framework
- Is itself subject to the invariants it defines — any proposed modification must pass IVE verification before it can be incorporated

The signatories below attest that they have reviewed this specification in its entirety, that it accurately represents the governance doctrine as understood within their respective domain of the D-3 Tri-Core, and that they authorize its elevation to Constitutional Artifact status.

Jon Adams Halstead

Architecture Domain — D-3 Tri-Core

Attest: Architecture continuity, structural doctrine fidelity, and layer relationship integrity of this specification.

Date: 6/17/2026

Dar-z

Governance Domain — D-3 Tri-Core

Attest: Governance continuity, mission boundary preservation, commercial coherence, and invariant completeness of this specification.

Date:

Daniel

Engineering Domain — D-3 Tri-Core

Attest: Engineering continuity, technical integrity, predicate correctness, and execution reliability of this specification.

Date:

AAES-OS Invariant Verification Engine Specification v1.0 — Document ID: AAES-OS-IVE-SPEC-001 — June 17, 2026

Classification: Canonical System Artifact — Constitutional Layer

This document becomes a Constitutional Artifact upon execution of all three signatures in Section 9.

© AAES-OS Constitutional Authorship Team — Jon, Dar-z, Daniel. All rights reserved under the AAES-OS governance framework.